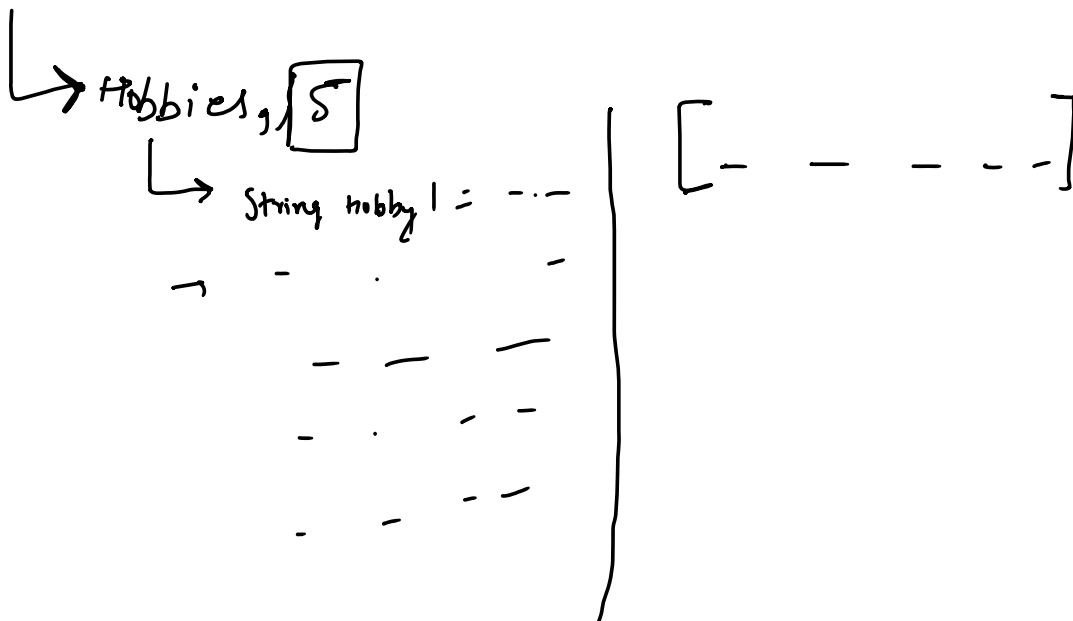


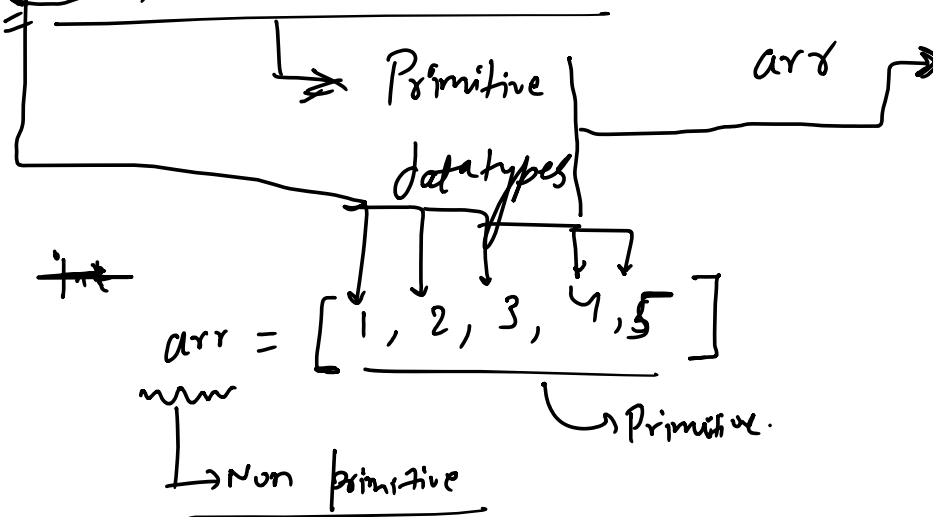
Arrays

24 March 2026

07:44



int, char, Boolean



→ int . [] . arr = { 1, 2, 3, 4, 5 } ;

↳ datatype

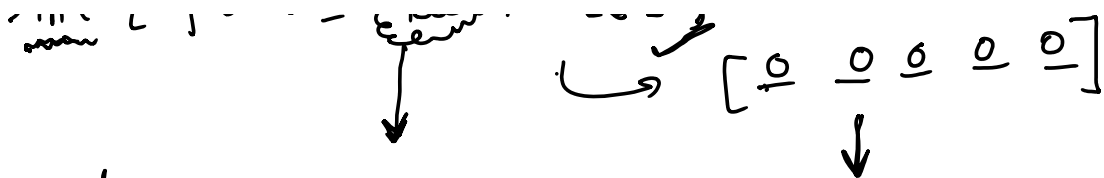
↳ array name

[1, 2, 3, 4, -1, -2 - - -]

→ [1, 2, 3, 4, 5]

→ int [] arr = new int [5] ;

↳ [0 0 0 0 0]



heap memory

→ Garbage value → (Hexadecimal value)

decimal → 0 - 9 2 3 4 1 F 3 2 0 9

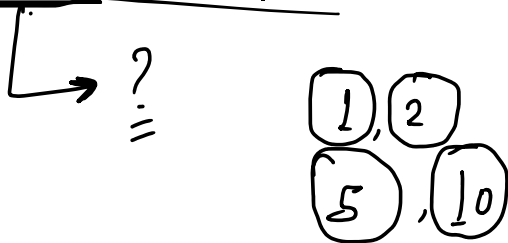
99 → 9999

0-9 A B C D E F

0, 1, 2, 3, 4, 5, 6, 7, 8, 9 A B C D E F

Name	→ String
Age	→ int
Hobbies	→

Heap Memory →



- 1 Sing.
- 2 Dance
- 3 painting
- 4 -
- 5 -
- 6 -

user input

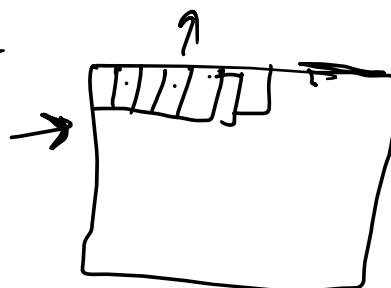
→ runtime

Sc.nextInt();

→ 7

new int [5]

new int [7] ✓



`int n = Sc.nextInt();`

↳ total elements of array

`int[] arr = new int[n];`

↳ [- - - - -];

$n \rightarrow$ runtime

$\rightarrow [1, 2, 3, 4, 5]$

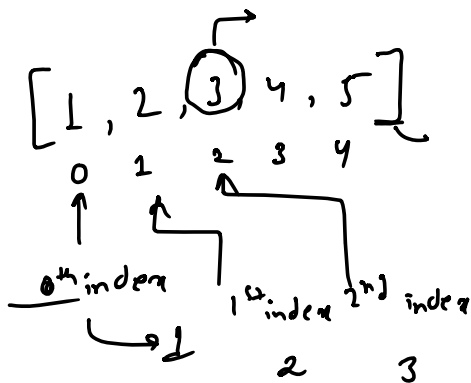
↳

[0, 0, 0, 0]

`int a = 5;`

`cout(a);`

`cout(array)`



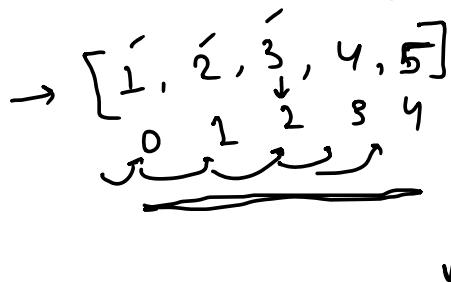
`cout(arr[2]);`
 ↳ index number

$0 \rightarrow$

`cout(arr[0]);`

↳

Size = 5 ($0 \rightarrow 4$)



`cout(arr[2]);`
 ↳ 3

Size = 5

$[0, \text{size}-1] = [0, \dots]$

rank $\rightarrow$ 1 2 3 4 5

arr → 1 2 3 4 5

$n = 5$

cout(arr[0])

cout(arr[1])

arr[2]

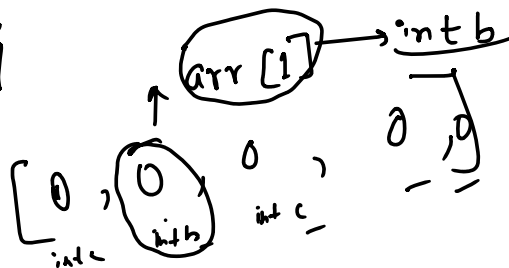
arr[3]

$[0, n-1]$ arr.length = Size max $i=4$ $i=0$

for(int $i=0$; $i < arr.length$; $i++$) {

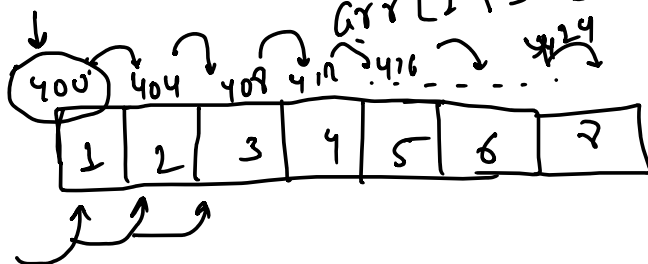
cout(arr[i]);

}



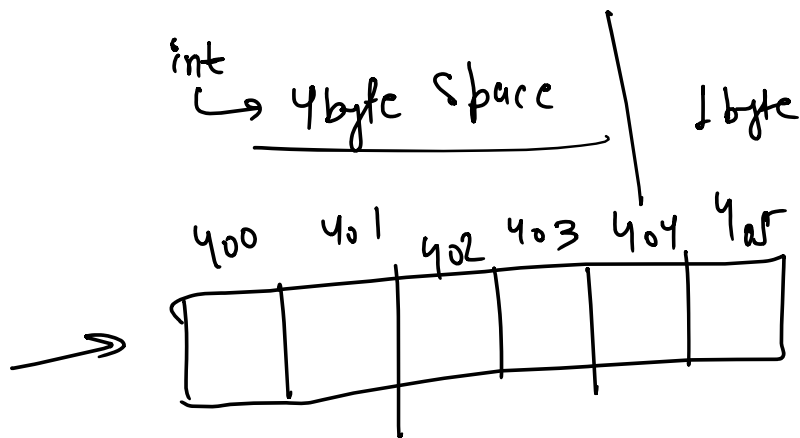
int b = sc.nextInt();

arr[1] = sc.nextInt();



400

Contiguous memory



Q. Array of size n (user i/p), & print the index of the largest element.

$[1, 4, 2, 3, 4]$
1

`int n = sc.nextInt();`

Op - 1

```
→ int [] arr2 = new int
    [n];
    for(int i =
    0; i < arr2.length; i++)
    {
        arr2[i] =
        sc.nextInt();
    }
```

$[0, 0, 0, 0]$
↑

$[1, 0, 0, 0]$
1

$[1, 4, 2, 3]$

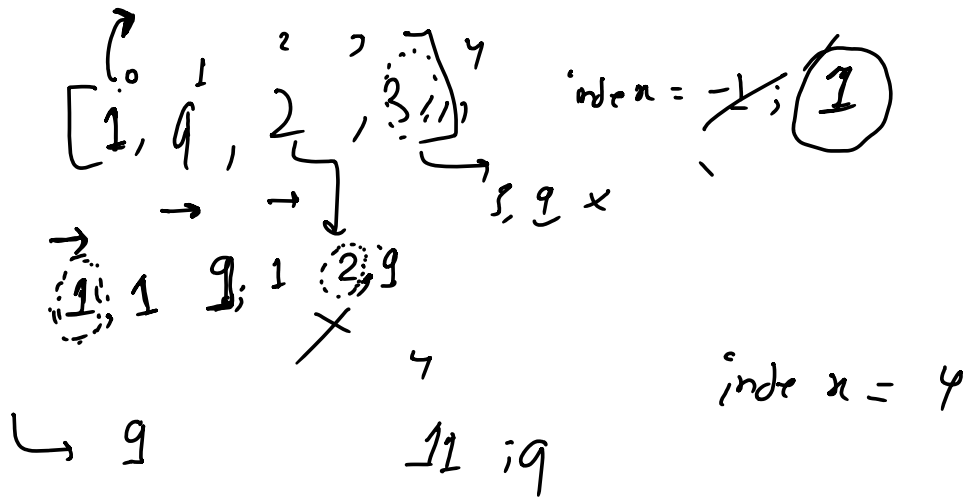
$[1, 4, 2, 3]$
1

```
int index = -1;
int num = arr[0];
for(int i = 0; i < arr.length; i++) {
```

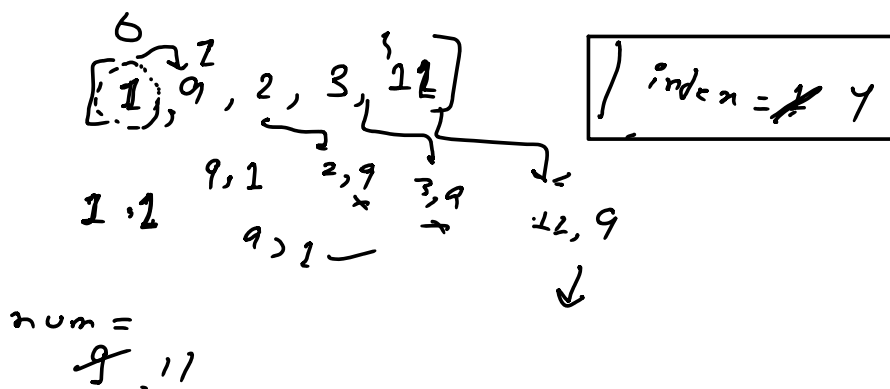
```

int num = arr[0];
for (int i = 0; i < arr.length; i++) {
    if (arr[i] > num) {
        index = i;
    }
}

```



Print (index);



Print (index);